```
    const weatherInfo = `The current temperature in ${city} is
${temperature}°C with ${weatherDescription}. The humidity level is
${humidity}%.`;
  const prompt = `Here is the latest weather update:
${weatherInfo}. How can I help you today?`;
```

Creates the prompt for the AI model by injecting the result of the weather info

Asks the AI model about the weather info

Although a simpler approach, this option is only useful when your application is serving prompts in a specific format. When your application consistently serves prompts in that way, injecting data into these prompts ensures that the format is maintained. It quickly falls apart if you want more flexibility in your responses, though, so you need to be aware of its limitations.

To recap, implementing tools and function calling in AI models offers several significant advantages. It enhances accuracy by providing access to up-to-date information, which is crucial for queries related to current events, real-time data, or rapidly changing fields. It also expands the AI's capabilities, enabling it to perform a wider range of tasks that extend beyond its initial training.

4.4.2 Implementing custom tools and functions with the Vercel AI SDK

The Vercel AI SDK makes it easy to include custom tools and functions. You simply define your custom functions using a Zod schema to specify parameters and then register them with the SDK. Here are the typical steps:

1. *Define your custom tools*. Create functions that represent the tools you want to make available to the AI model. These functions should accept specific parameters and return structured data using the Zod validation library.
2. *Register tools with the SDK*. Use the SDK's tools method to register your custom tools, specifying their names, parameters, and descriptions.
3. *Handle tool response rendering*. Implement logic to render the response of the tool back as React components.

Let's use a simplified example of a weather information tool. I'll explain both the prompt and the tool's parameter.

Here's what we're going to build:

- A weather assistant AI that can provide real-time weather information for specific cities, demonstrating the integration of custom tools with the Vercel AI SDK
- A custom tool that fetches weather data and returns it as a structured React component
- An updated `streamUI` function for rendering of AI responses and tool outputs, including loading states and final weather information displays

In listing 4.13, we first present a proper system message that defines the AI's role as a weather assistant. It instructs the AI to use the `getWeather` function when asked about